VAE vs DDPM: Implementing VAE and DDPM architectures with comparison and models differences

GenCV003 - CV Engineer Assignment

[Nouran Khatab]

[March 16th 2026]

1. Introduction

This report presents my story with the 'from-scratch' implementation and comparison of two fundamental/basic deep generative models in the field of Computer Vision: Variational Autoencoders (VAE) and Denoising Diffusion Probabilistic Models (DDPM). It aims to be a summary of the process, not drowning in the mathematical details (the basic details are planned to be in the supplementary video, mainly using the hand-written notes). It doesn't aim to be a reference on the two models, for actual reading, you can check the Recommended Resources section)

2. Background & Theory

2.1 Variational Autoencoder (VAE)

VAEs are built on the idea of regular AutoEncoders (using an encoder network to create a laten representation (Z space) of the distribution of the input data (*automatically*, without imposing our representation/perception of the data), then a decoder network to Reconstruct the original image out of its corresponding point in Z space. VAEs special addition is creating the latent space, not only out of the predictions of the encoder, but with also an additional 'stochastic' part, sampled from an independent distribution (gaussian with RGB data). Implementation wise, stitching was straightforward with an addition of experimenting with different beta values, for the sake of debugging and testing its effect.

- Encoder maps *x* to distribution parameters (*mu*, *logvar*)
- Reparameterization trick: *z* = *mu* + *sigma* * *epsilon*
- Decoder maps z back to image
- Loss = Reconstruction + beta * KL Divergence
- beta controls the recon vs latent structure trade-off

2.2 Denoising Diffusion Probabilistic Model (DDPM)

DDPM is what enabled us to use the concept of diffusion generative DNNs in large-scale high-quality results in image "generation". (Ehm, with strong GPUs and high res. images). To implement them, the first that came to mind is that we need to stitch together the NoiseSchedule and the SimpleUNet and sample as per 'orignal' paper, not the enhanced ones (at first). But then faced mainly 2 problems, using 'integer'/1-D timesteps vector as inputs to the NN hinders its ability to extract complex meaningful features, found out that other implementations uses this type of embedding (the idea is similar to positional encoding in the original transformer)

- Forward process: gradually add noise over T steps
- Can jump to any noise level: x_t = sqrt(alpha_bar) * x_0 + sqrt(1-alpha_bar) * noise
- U-Net predicts the noise given noisy image, timestep (as if we're 'SEGMENTING' the noise)
- Loss = MSE(predicted noise, actual noise)
- Sampling: start from noise, iteratively denoise (Langevin Dynamics!!)

2.3 Key Theoretical Differences

Aspect	VAE	DDPM
Network learns	Compress + decompress	Predict noise
Loss	Recon + beta*KL	MSE on noise
Sampling speed	1 forward pass	T iterations (slow)
Sample quality	Blurry	Sharper
Latent space	Explicit	No explicit latent space
Training	ELBO	Simple denoising

3. Implementation Details

3.1 Dataset

- CIFAR-10 (32x32 RGB), filtered to 3 classes: plane, dog, ship
- Training: 10,000 samples | Test: 2,000 samples
- Preprocessing: ToTensor() to [0, 1] [affects clipping]

3.2 VAE Architecture

Hyperparameter	Value
Latent dim	128
Base channels	32
Encoder	4x Conv2d stride 2
Decoder	4x ConvTranspose2d
Optimizer	Adam lr=1e-3
Batch size	128
Epochs	100

3.3 DDPM Architecture

Hyperparameter	Value
Timesteps	1000
Schedule	Linear beta 1e-4 to 0.02
U-Net channels	32
Time embedding	Sinusoidal dim=128
Optimizer	Adam lr=1e-4
Batch size	64
Epochs	100

3.4 Challenges & Debugging

3.4.1 VAE

- Visualizing the Z space (even with using manifold-based dimensionality reduction) didn't produce intuitive results, I wanted to see more clustered classes, to experiment with concept vectors (testing same class across feature vector Z1 while keeping other Zj vectors the same)

It probably needed a simpler dataset, like the shapes one in deepbean video.

3.4.2 DDPM

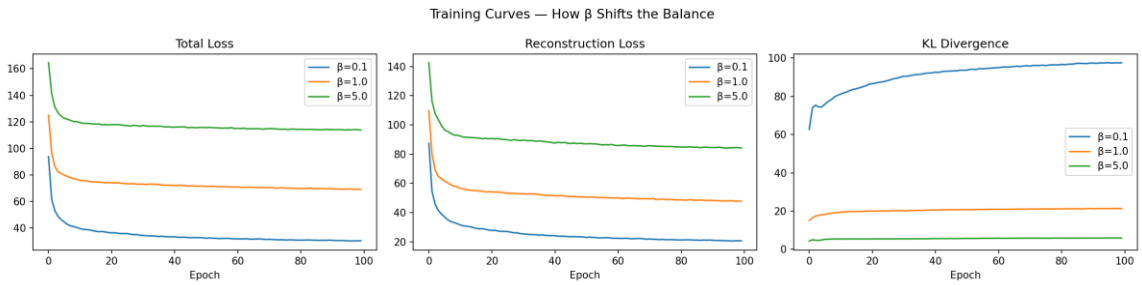
1. Using 'integer'/1-D timesteps vector as inputs to the NN hinders its ability to extract complex meaningful features, found out that other implementations uses this type of embedding (the idea is similar to positional encoding in the original transformer), so implemented it with Claude.
2. Not injecting 'fresh stochasticity' in the sampling part
   Imagine that finally, after the training process and heading to sampling to see the magic, you're hit with this

   It turned out it's the tendency of the Unet to produce a slop of the mean values of the images, instead of a sha
   The solution was to inject fresh noise for each timestep (except for the last one in the reverse order, AKA the final generated image)
   It also turned out that it was related to physics concepts (Langevin Dynamics, Brownian motion)
   But it also makes me wonder, does all the generations without this part product the same slop?

4. Results

4.1 Training Curves

4.1.1 VAE across 3 beta values

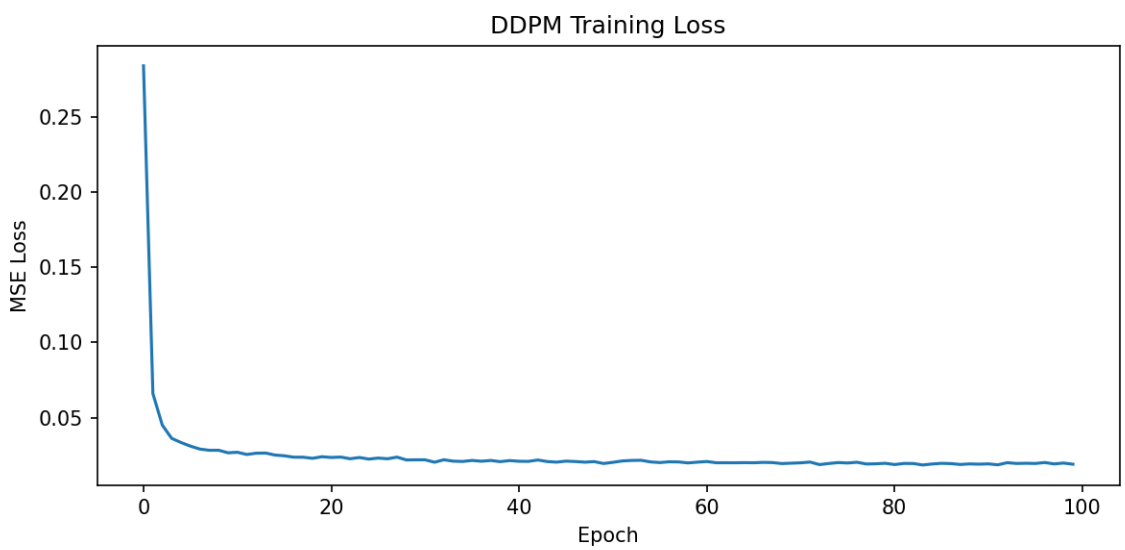


As expected, lower beta values lead the model to completely ignore KL because it's barely penalized (high KL loss) which allows the reconstruction power to flourish (least reconstruction loss).

Higher beta values force the encoder to compress into a tighter, more gaussian latent space, which hurts reconstruction quality (more hurt -> more loss)

Total loss is dominated by reconstruction -> follows its trend

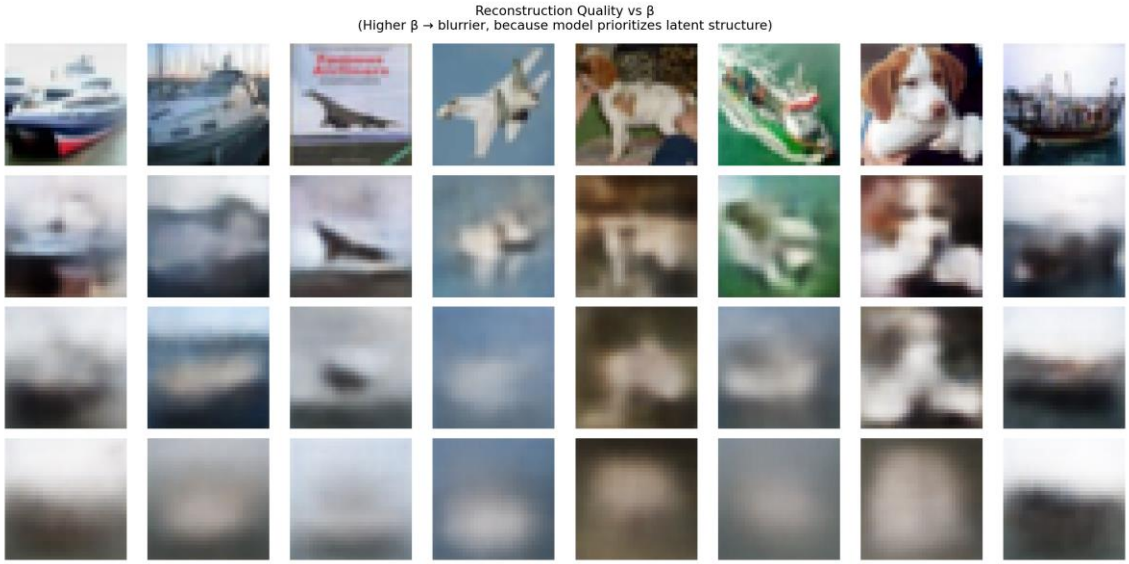
4.1.2 a 100-epochs DDPM



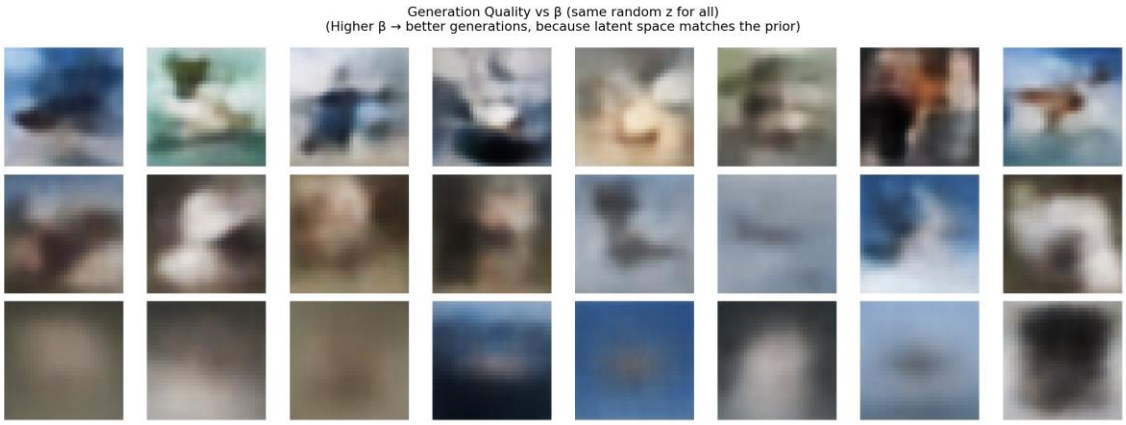
clean, stable convergence, turned out that this training stability is one of the key practical advantages of diffusion models over GANs and VAEs: there are no mode collapse issues, no beta tuning, etc. This also proves the model's ability to learn, as a way of debugging the implementation.

4.2 Beta-VAE Trade-off

(beta moves from 0.1 to 1 to 5)



Higher beta values leads to (1) more regularized latent space (2) stronger push of the posterior dist. to resemble the normal gaussian one. Using communication flow terms, they reduce the number of bits of information that can flow from encoder to decoder through the latent vector z, because a distribution that is close to a unit gaussian has low KL divergence but also low mutual information I(x; z).



First row is (by definition) the worst generations, barely controlling the z space, leading to z to land in regions the decoder was never trained on. Last row is also a typical high beta value behavior, largely smoothed (excessively in a way as well)

4.3 Generated Samples: VAE vs DDPM

In VAEs, we test the model's ability to reconstruct a meaningful image out of a random z in the 'continuous' and 'complete' latent space, not its ability to 'restore/remember' a specific set of images (a regular AE would already do that)

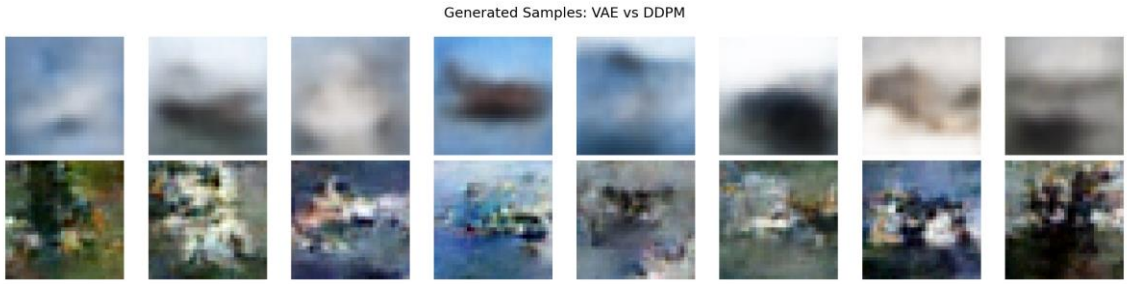
In DDPMs, the model is starting from an 'information-less' noisy image from a dataset, and it denoises it using the trained Unet on the same dataset, and at some point, it starts to resemble the 'group of images' it belongs to in the original dataset (that groups has mu and std)

4.4.1 Qualitative analysis

Can't use the 'same images' as DDPMs start from a 'randomly' noised image

Phenomena -> VAE images are more washed out and blurry, they also contain less details, while DDPM images are sharper with more elements but it's not clear to which 'group of images' they belong to

Justification -> [the Discussion part]



4.4.2 Quantitative Metrics

Metric	VAE (beta=1.0)	DDPM
FID (gen) lower=better	288.1564	260.8518
KID (gen) lower=better	0.1764 ± 0.0087	0.1527 ± 0.0078

Frechet Inception Distance (FID) (used instead of IS as it's a better enhancement on it) mathematically requires a large batch size (~50,000 samples) to calculate an unbiased covariance matrix. Running FID on only 100 generated samples (due to CPU limits) provides a *relative* comparison between the two models, rather than an absolute benchmark of standard image quality. KID was tested as a lighter alternative for small samples, but it took longer time in calculating (interesting)

Both metrics answer the same question: "How similar are the generated images to the real images, as a group?" They don't compare image-to-image. They compare the distribution of generated images vs real images.

As expected, DDPM scores are better, they produce sharper 'closer' images to the original distribution, and it was also expected to have high FID values because both of them produce images that doesn't resemble the original distribution when evaluated with eye (needs more training epochs or more complex backbones)

5. Discussion

It's clear that absolute/individual comparisons would be hard, the generated images alone don't tell clearly the specific details of their respective models, but we can do it in a relative/subjective manner.

First, comparing sharpness and blurriness. VAE in this case is characterized by having (1) prior distribution assumed to be a unit gaussian averaging one and (2) inherited mathematical tension between the reconstruction factor (pushing towards shaper reconstructions but producing unregularized latent space) and KL Divergence (pushing towards that gaussian dist.). On the other hand, DDPMs do not have a latent space bottleneck or a KL tradeoff, the U-Net pushes pixels toward sharp, distinct, high-probability manifolds without having to average out intersecting latent spaces.

Another point of discussion is the performance under limited resources. Diffusion models are known to be computationally expensive requiring long hours (days) of training with a repeated process of O(T) complexity but also producing sharper images. While VAEs are known to be much less computationally expensive (relatively) while producing washed out images (or sharp ones with the cost of irregular latent space) with O(1) complexity. It's the classical tradeoff between resources consumption and the output quality, and it's a collective decision to take when discussing with other teams in the company.

I'd also like to mention was introduced to idea of using heatmaps on the error parts in the image (like which parts were hard to generate) but it didn't feel right in this context, maybe for a business context of high resolution images that needs a specific semantic/concept generation to not go wrong/different from the original image (more than other parts) would be a use case for this idea.

6. Reflection

This has been an amazing learning experience (wished to be able to dedicate more time to it), regardless of not achieving near SOTA results. Given that my only previous experience with generative models didn't span the visual data, and constraints of not using full pre-existing implementations, I feel proud enough with what I achieved & learned in the time frame of 3 days. It was also a reminder of my weakest points in coding that I find myself heading fast to Claude (the visualization syntax, kind of med-level python knowledge like decorator, and SWE-oriented code). I usually convince myself that it's better to focus on the architecture as a whole, understanding the theoretical concept (given that they're good implementers) but I also believe that the tool is as good as its master, and I can't unlock their full potential when what I know is less and it's for my long-term benefit to use it mainly in the 'teaching'/'consultation' mode (more in this and this)

7. Recommended Resources

1. An introductory video to the topic of Generative DNNs by MIT, no fancy math: https://www.youtube.com/watch?v=SdTZAMDKrNY&list=PLtBw6njQRU-rwp5__7C0oIVt26ZgjG9NI&index=5
2. The two lectures on the topic by the great lecturer Justin Johnson (old but gold, a comfort course on the CV basics), he didn't cover the diffusion here https://www.youtube.com/watch?v=Q3HU2vEhD5Y&list=PL5-TkQAfAZFbzxjBHtzdVCWE0Zbhomg7r&index=20

7.1 VAEs

1. The original paper: https://arxiv.org/abs/1312.6114
2. [The most details inspiring, watched more than one time] A great video, not recommended as a very first encounter with the topic, but it's concentrated in a great way: https://www.youtube.com/watch?v=HBYQvKlaE0A and its sequel on β-VAE
3. A blog post to view VAEs from both DL and Probability perspectives, I found it very similar to the videos above https://jaan.io/what-is-variational-autoencoder-vae-tutorial/
4. [Couldn't use due to assessment constraints] Building a VAE from scratch using PyTorch live https://www.youtube.com/watch?v=Nn20SUGY6qs
5. It was highly recommended by my track supervisor, didn't watch it but still on the list to check it out because she encouraged me to do so https://www.youtube.com/watch?v=gOIXGhW0VIs

7.2 DDPM

I didn't test one source from start to finish as with VAEs, like whenever I felt like I need more information about a certain point, I look for a couple of minutes/paragraphs from another resource, and so on

Doing and watching the code was most helpful in this back and forth process

1. The original paper: https://arxiv.org/abs/2006.11239
2. https://huggingface.co/blog/annotated-diffusion Until before code
3. https://www.youtube.com/watch?v=81l9b0CaS-A A short concept-focused lecture
4. https://www.youtube.com/watch?v=sVkfcaAvHgQ A mental drill on DDPM math. details (same channel as 7.1.2), recommended after concepts 'working/good enough' understanding
5. https://lilianweng.github.io/posts/2021-07-11-diffusion-models/ Lilian Blogs are always deep, comprehensive and mentally satisfying, didn't read it yet, but it should definitely be in the resources I'm getting back to when having more time.